

Micro-ERP Auto

Estrutura de Commits e Modelo de Dados

Projeto:	Micro-ERP Auto (Software 2 - Projeto Academico)
Organizacao:	Auto-academic-erp
Repositorio:	github.com/Auto-academic-erp/micro-erp-auto
Pull Request:	#1 - feature/backend-import-initial → develop
Documento:	docs/ESTRUTURA_COMMITS_E_SCHEMA.pdf
Data:	11 de maio de 2026
Autor da sessao:	Diego Mendes Santos (coordenando) com Luan Gonzaga (autor do codigo)

Este documento detalha (1) a sequencia de commits adotada para a importacao inicial do backend C# desenvolvido pelo Luan Gonzaga, (2) o significado e comportamento de cada campo das 8 entidades do modelo de dados, e (3) o sumario do security review automatizado executado sobre o PR #1, incluindo o plano de correcao do unico achado de severidade alta.

Sumario

- 1. Estrutura dos 6 commits do PR #1
- 2. Modelo de dados - comportamento dos campos por entidade
 - 2.1 Usuario
 - 2.2 Configuracao
 - 2.3 Cliente
 - 2.4 Atendimento
 - 2.5 Produto
 - 2.6 Servico
 - 2.7 ItemProduto
 - 2.8 ItemServico
- 3. Convencoes transversais (timestamps, offline-first, multi-tenant)
- 4. Sumario do security review do PR #1
- 5. Plano de correcao do achado HIGH

1. Estrutura dos 6 commits do PR #1

A importacao do backend foi organizada em 6 commits sequenciais (ao inves de um unico commit gigante) para que o historico do repositorio conte uma narrativa coerente, facilite revisao incremental e permita reverter fases especificas se necessario. Cada commit foi pensado para **compilar isoladamente** (build verde apos cada um), evitando o anti-padrao de commits intermediarios quebrados.

Todos os commits sao autorados como **Luan Gonzaga Oliveira <luangonzagaoliveira@gmail.com>** (autor original do codigo), embora tenham sido organizados em fases na maquina do Diego durante o onboarding ao GitHub.

1.1 Visao geral dos 6 commits

1	6055120	chore	Scaffold inicial do projeto ASP.NET Core 8	KAN-13
2	d8f2c42	feat	Modelo de dados - 8 entidades EF Core e ApplicationDbContext	KAN-19
3	6a472d7	feat	Autenticacao JWT com cadastro e login de Usuario	KAN-20
4	51b0d87	feat	CRUDs Cliente, Produto, Servico, Atendimento, Itens	KAN-22 a 26
5	2ab99d6	chore	Arquivo HTTP smoke test e respostas de validacao padronizadas	-
6	68c8c98	chore(ci)	Habilita Dependabot para NuGet e GitHub Actions	-

1.2 Detalhamento por commit

Commit 1 — chore: scaffold inicial ASP.NET Core 8 (KAN-13)

Hash: 6055120 Arquivos: 6 novos Linhas: +140

Conteudo:

- **MicroERP.sln** — solution Visual Studio referenciando o projeto MicroERP.Api
- **MicroERP.Api/MicroERP.Api.csproj** — projeto Web API, TargetFramework net8.0, Nullable enable, ImplicitUsings enable. Dependencias: EntityFrameworkCore 8.0.26, Npgsql.EntityFrameworkCore.PostgreSQL 8.0.4, Microsoft.AspNetCore.Authentication.JwtBearer 8.0.26, Swashbuckle.AspNetCore 6.6.2
- **MicroERP.Api/Program.cs** — versao minimal contendo apenas Controllers, Swagger e HttpsRedirection
- **MicroERP.Api/appsettings.json** — placeholders vazios (sem secrets) para ConnectionString e Jwt
- **MicroERP.Api/appsettings.Development.json.example** — template com instrucoes para cada dev preencher localmente (NAO vai pro git por estar no .gitignore)
- **MicroERP.Api/Properties/launchSettings.json** — perfis de execucao IIS e Kestrel (https://localhost:5001)

Por que primeiro? Estabelece a fundacao do projeto sem qualquer dependencia ou logica de negocio. Garante que dotnet build compila com sucesso antes de adicionar qualquer codigo.

Commit 2 — feat: modelo de dados (8 entidades + ApplicationDbContext)

Hash: d8f2c42 Arquivos: 10 novos Linhas: +311

- **Models/Usuario.cs, Configuracao.cs, Cliente.cs, Atendimento.cs, Produto.cs, Servico.cs, ItemProduto.cs, ItemServico.cs** — as 8 entidades do DER (PROJETO_CONTEXTO secao 5)
- **Data/AppDbContext.cs** — DbContext com Fluent API: HasMaxLength em strings, ColumnType decimal(12,2) em monetarios, indices unicos (Email, Cpf, Uuid), indices em Usuariold, relacionamentos com OnDelete Cascade ou Restrict conforme o caso

- **Program.cs (atualizado):** registra `AddDbContext<AppDbContext>` com `Npgsql`

Por que aqui? Modelos são a base sobre a qual tudo o que vem depois (Auth, CRUDs) depende. Como não referenciam serviços externos, este commit isolado já deve compilar sem erros.

Commit 3 — feat: autenticação JWT e cadastro de Usuário (KAN-20)

Hash: 6a472d7 Arquivos: 8 novos + 1 modificado Linhas: +272

- **Controllers/AuthController.cs** — POST /api/auth/register e POST /api/auth/login
- **Services/AuthService.cs + Services/Interfaces/IAuthService.cs** — hash de senha com PasswordHasher<Usuario> (PBKDF2 + HMAC-SHA512 + salt) e geração de JWT (HMAC-SHA256, claims sub/email/jti)
- **Services/Exceptions/EmailAlreadyExistsException.cs** — retorna 409 Conflict
- **DTOs/AuthRegisterRequest.cs, AuthLoginRequest.cs, AuthResponse.cs** — contratos dos endpoints
- **DTOs/ApiResponse.cs** — envelope padrão de resposta (incluído aqui porque AuthController já o usa)
- **Program.cs (atualizado)**: AddAuthentication/JwtBearer, AddAuthorization, registra IAuthService no DI, middlewares UseAuthentication e UseAuthorization

Por que antes dos CRUDs? Os controllers dos CRUDs dependem de [Authorize] e da extração da claim sub (Usuariold). Sem JWT pronto, os CRUDs não poderiam ser commitados de forma funcional. Também alinha com a entrega da KAN-20 que estava marcada como concluída pelo Luan.

Commit 4 — feat: CRUDs Cliente/Produto/Serviço/Atendimento/Itens (KAN-22 a 26)

Hash: 51b0d87 Arquivos: 50 novos + 1 modificado Linhas: +2.239

- **6 Controllers** (Cliente, Produto, Serviço, Atendimento, ItemProduto, ItemServiço) com [Authorize] e extração de Usuariold da claim sub
- **6 Services + 6 Interfaces** — lógica de negócio (validação, cálculo de subtotal, calcularTotal do Atendimento)
- **6 Repositories + 6 Interfaces** — acesso a dados via EF Core, filtrando por Usuariold (multi-tenant)
- **Services/Exceptions/CpfAlreadyExistsException.cs** — conflito de CPF
- **18 DTOs** (3 por entidade: Create, Update, Response)
- **Program.cs (atualizado)**: 12 chamadas AddScoped para registrar Services e Repositories no DI

Por que tudo junto? As 6 entidades operacionais foram entregues como um bloco porque compartilham o mesmo padrão (Controller → Service → Repository) e foram desenvolvidas em paralelo pelo Luan. Separa-las em 6 commits adicionaria ruído sem ganho de clareza.

Commit 5 — chore: arquivo HTTP smoke test e validação padronizada

Hash: 2ab99d6 Arquivos: 1 novo + 1 modificado Linhas: +68

- **MicroERP.Api/MicroERP.Api.http** — requests prontas para teste manual no Visual Studio / VS Code (register, login, listar clientes, produtos, serviços, atendimentos com bearer token)
- **Program.cs (atualizado)**: ConfigureApiBehaviorOptions que intercepta erros de ModelState (DTO inválido) e responde com ApiResponse padronizado (Success=false, Message, Errors), garantindo formato consistente em toda a API

Commit 6 — chore(ci): habilita Dependabot

Hash: 68c8c98 Arquivos: 1 novo Linhas: +30

- **.github/dependabot.yml** — varredura semanal de pacotes NuGet (segunda 08:00 SP, max 5 PRs), varredura mensal de actions do CI
- Reviewers automáticos: time Auto-academic-erp/core-devs
- Labels automáticas: dependencies + backend ou ci

Por que último? Configuração de CI/segurança é ortogonal ao código. Mantém o commit isolado e auditável.

2. Modelo de dados - comportamento dos campos

As 8 entidades implementam o DER especificado no PROJETO_CONTEXTO.md secao 5. Para cada campo, esta secao indica: (a) significado, (b) tipo SQL no PostgreSQL, (c) se participa do mecanismo offline-first, (d) se participa do isolamento multi-tenant, (e) quando e por quem o campo eh populado.

2.1 Usuario - tabela Usuarios

Conta de microempreendedor (tenant). Toda outra entidade operacional referencia esta via `UsuarioId`. Nao tem campos offline-first porque o usuario se autentica online no primeiro uso e o token JWT identifica o tenant nas requests subsequentes.

Id	BIGINT PK	-	Auto	Banco gera no INSERT (sequence)
Nome	VARCHAR(120) NOT NULL	-	-	AuthRegisterRequest no cadastro
Email	VARCHAR(150) UNIQUE	-	-	AuthRegisterRequest. Unique global.
Senha	VARCHAR(255)	-	-	AuthService.RegisterAsync com PasswordHasher (PBKDF2)
CreatedAt	TIMESTAMP DEFAULT NOW()	-	-	Default UTC no momento do RegisterAsync
UpdatedAt	TIMESTAMP	-	-	Atualizado quando o usuario edita perfil (endpoint futuro)

2.2 Configuracao - tabela Configuracoes

Relacionamento 1:1 com Usuario. Define o **modo de operacao do sistema** para aquele tenant: Venda (so produtos), Servico (so servicos) ou Hibrido (ambos). Esta entidade eh o "cerebro" do sistema — o Frontend e o Mobile leem dela para decidir quais modulos exibir. **OBS:** o codigo atual do Luan tem apenas o Model; falta o Controller/Service/Repository (KAN-21 em aberto).

Id	BIGINT PK	-	-	Banco gera no INSERT
TipoOperacao	VARCHAR(10) NOT NULL	-	-	Tela de parametrizacao no Frontend (UC03)
UsuarioId	BIGINT FK UNIQUE	-	Sim	1:1 com Usuario. Unique garante uma config por tenant.
CreatedAt	TIMESTAMP	-	-	Default UTC
UpdatedAt	TIMESTAMP	-	-	Atualizado quando o usuario muda o modo

2.3 Cliente - tabela Clientes

Cadastro de clientes do microempreendedor. **Primeira entidade com campos offline-first** (UUID, SyncedAt, DeletedAt), porque o app mobile precisa cadastrar clientes sem internet.

Id	BIGINT PK	-	-	Banco gera no INSERT
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera localmente antes do sync. Garante unicidade sem servidor.
Nome	VARCHAR(120) NOT NULL	-	-	Usuario via formulario
Telefone	VARCHAR(20) NULL	-	-	Usuario via formulario (opcional)
Cpf	VARCHAR(14) UNIQUE	-	-	Usuario via formulario. Vide secao 5: unicidade deve ser composta com Usuariold.
Usuariold	BIGINT FK	-	Sim	Backend extrai do JWT claim sub
CreatedAt	TIMESTAMP	-	-	Default UTC
UpdatedAt	TIMESTAMP	-	-	Atualizado a cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente de sync. Backend preenche no INSERT/UPDATE; sync mobile atualiza apos confirmacao.
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete. Replicado bidirecionalmente no sync.

2.4 Atendimento - tabela Atendimentos

Registro central da operacao. Um Atendimento pode conter simultaneamente N ItemProduto e N ItemServico (Regra 2 - Atendimento Hibrido).

Id	BIGINT PK	-	-	Banco gera
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera localmente
DataRegistro	TIMESTAMP NOT NULL	-	-	Cliente envia ou backend usa NOW()
Status	VARCHAR(30) NOT NULL	-	-	Aberto / Em andamento / Concluido / Cancelado
ValorTotal	DECIMAL(12,2)	-	-	Calculado pelo Service (soma subtotais de itens)
Usuariold	BIGINT FK	-	Sim	Backend via JWT
Clienteld	BIGINT FK	-	-	Cliente envia (deve pertencer ao mesmo Usuariold)
CreatedAt	TIMESTAMP	-	-	Default UTC
UpdatedAt	TIMESTAMP	-	-	A cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete

2.5 Produto - tabela Produtos

Item fisico de inventario. QuantidadeEstoque eh decrementada (no Service) quando ItemProduto eh criado no contexto de um Atendimento.

Id	BIGINT PK	-	-	Banco gera
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera
Nome	VARCHAR(120) NOT NULL	-	-	Usuario via form
Preco	DECIMAL(12,2) NOT NULL	-	-	Usuario define no cadastro
QuantidadeEstoque	INT NOT NULL	-	-	Inicial: usuario define. Atualizado pelo ProdutoService nos lancamentos.
Usuariold	BIGINT FK	-	Sim	Backend via JWT
CreatedAt	TIMESTAMP	-	-	Default
UpdatedAt	TIMESTAMP	-	-	A cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete

2.6 Servico - tabela Servicos

Catalogo de servicos com valor por hora. Diferente de Produto, nao tem controle de estoque (servico nao se esgota).

Id	BIGINT PK	-	-	Banco gera
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera
Descricao	VARCHAR(200) NOT NULL	-	-	Usuario define (ex.: "Corte de cabelo")
ValorHora	DECIMAL(12,2) NOT NULL	-	-	Usuario define preco/hora
Usuariold	BIGINT FK	-	Sim	Backend via JWT
CreatedAt	TIMESTAMP	-	-	Default
UpdatedAt	TIMESTAMP	-	-	A cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete

2.7 ItemProduto - tabela ItemProdutos

Linha de produto vendido em um Atendimento. O Subtotal eh calculado no Service como Quantidade x PrecoUnitario e somado ao ValorTotal do Atendimento.

Id	BIGINT PK	-	-	Banco gera
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera
Quantidade	INT NOT NULL	-	-	Usuario via form (numero de unidades)
PrecoUnitario	DECIMAL(12,2)	-	-	Service preenche com Produto.Preco no momento do lancamento (snapshot historico)
Subtotal	DECIMAL(12,2)	-	-	Service calcula: Quantidade x PrecoUnitario
AtendimentoId	BIGINT FK	-	Indireto	Heranca do Atendimento.UsuarioId. ON DELETE RESTRICT.
ProdutoId	BIGINT FK	-	Indireto	Heranca do Produto.UsuarioId. ON DELETE RESTRICT.
CreatedAt	TIMESTAMP	-	-	Default
UpdatedAt	TIMESTAMP	-	-	A cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete (nao aplicado no diff atual mas previsto)

2.8 ItemServico - tabela ItemServicos

Linha de servico prestado em um Atendimento. Estrutura analoga a ItemProduto. Quantidade representa horas trabalhadas (ou unidades de servico).

Id	BIGINT PK	-	-	Banco gera
Uuid	CHAR(36) UNIQUE	Sim	-	Mobile gera
Quantidade	INT NOT NULL	-	-	Usuario via form (horas)
PrecoUnitario	DECIMAL(12,2)	-	-	Service preenche com Servico.ValorHora (snapshot)
Subtotal	DECIMAL(12,2)	-	-	Service calcula: Quantidade x PrecoUnitario
AtendimentoId	BIGINT FK	-	Indireto	Heranca do Atendimento.UsuarioId
ServicoId	BIGINT FK	-	Indireto	Heranca do Servico.UsuarioId
CreatedAt	TIMESTAMP	-	-	Default
UpdatedAt	TIMESTAMP	-	-	A cada UPDATE
SyncedAt	TIMESTAMP NULL	Sim	-	NULL = pendente
DeletedAt	TIMESTAMP NULL	Sim	-	Soft delete

3. Convenções transversais

3.1 Timestamps (CreatedAt / UpdatedAt)

Toda entidade tem `CreatedAt` e `UpdatedAt` em UTC. `CreatedAt` é imutável após o `INSERT`. `UpdatedAt` deve ser atualizado pelo Service a cada `UPDATE` (no PR atual a atualização explícita está delegada ao código do Service).

3.2 Mecanismo offline-first (Uuid, SyncedAt, DeletedAt)

São três campos que viabilizam o app mobile funcionar sem internet e sincronizar quando voltar online. Funcionamento:

- **Uuid (CHAR 36) UNIQUE:** gerado pelo app mobile *antes* de qualquer chamada ao servidor. Garante identificação única do registro mesmo enquanto o BIGINT Id do servidor não existe.
- **SyncedAt (TIMESTAMP NULL):** NULL indica que o registro ainda não foi sincronizado. Após sync bem-sucedido com o servidor, o app preenche com o instante do envio.
- **DeletedAt (TIMESTAMP NULL):** implementa soft delete. O registro permanece fisicamente na tabela mas é tratado como removido. Replicado bidirecionalmente: se o mobile deleta um registro offline, o servidor preserva o `DeletedAt` e até outros devices podem ver a remoção.
- **Resolução de conflitos:** regra *last write wins* baseada em `UpdatedAt` (PROJETO_CONTEXTO seção 11, Regra 3).

3.3 Multi-tenant (UsuarioId)

Toda entidade operacional (Cliente, Atendimento, Produto, Serviço) carrega `UsuarioId` apontando para a conta do microempreendedor. Itens (`ItemProduto`, `ItemServiço`) herdam o tenant indiretamente via `AtendimentoId`. **Regra 4 do PROJETO_CONTEXTO:** toda query EF Core deve filtrar por `UsuarioId`. O `UsuarioId` é extraído do JWT (claim `sub`) no Controller e passado adiante para o Service. Nunca confiar em `UsuarioId` vindo do request body.

3.4 Tipos monetários

Todos os valores monetários usam `DECIMAL(12, 2)` — 10 dígitos antes da vírgula e 2 depois — jamais `float` ou `double`. Isso evita erros de arredondamento financeiro (ex.: $0.1 + 0.2 \neq 0.3$ em `float`).

4. Sumario do Security Review do PR #1

Foi executado o security review automatizado sobre o diff completo do PR #1. O processo seguiu 3 fases: identificacao por sub-agente, validacao paralela de cada finding contra os filtros de false-positive, e descarte de findings com confianca abaixo de 8/10.

Metrica	Valor
Findings reportados	3
Apos filtro de false-positives	1
Severidade do confirmado	1 HIGH (confianca 9/10)
Categoria	Authorization bypass / IDOR / Multi-tenant breach
Bloqueia merge?	Sim — corrigir antes de mergear em develop

4.1 Achado HIGH — CPF enumeration cross-tenant

Arquivos: Repositories/ClienteRepository.cs:39-44, Services/ClienteService.cs:33 e :67, Controllers/ClienteController.cs:82 e :111.

Descricao: ClienteRepository.GetByCpfAsync(string cpf) consulta a tabela Clientes sem filtrar por UsuarioId, retornando o primeiro match em qualquer tenant. ClienteService.CreateAsync e UpdateAsync chamam essa busca e lancam CpfAlreadyExistsException se receberem qualquer resultado, gerando 409 Conflict. Viola a Regra 4 (multi-tenant isolation) e cria um oraculo de enumeracao de CPF.

Exploit scenario:

POST /api/cliente com bearer token do tenant A enviando CPF arbitrario.

- 201 Created ⇒ CPF nao existe em nenhum tenant.
- 409 Conflict ⇒ CPF existe em ALGUM tenant.

Resposta deterministica permite enumeracao de CPFs em massa. CPF eh PII sob LGPD.

Avaliacoes positivas observadas (informativo, fora do escopo do achado):

- Senha hashada via PasswordHasher (PBKDF2 + HMAC-SHA512 + salt) em AuthService.cs:47
- Login retorna 401 generico sem vaziar quais emails existem
- JWT validation completa: ValidateIssuer, ValidateAudience, ValidateLifetime, ClockSkew 2 min
- RequireHttpsMetadata = true no JwtBearer
- Todos os Controllers exceto Auth tem [Authorize]
- UsuarioId extraido do JWT (claim sub), nunca do request body
- Demais Services/Repositories filtram corretamente por UsuarioId
- Campos offline-first (Uuid, SyncedAt, DeletedAt) presentes em todas operacionais
- AsNoTracking() em queries de leitura
- appsettings.json sem secrets (placeholders vazios)

5. Plano de correcao do achado HIGH

Correcao minima envolve 4 arquivos e aproximadamente 10 linhas de codigo. Sera entregue como novo commit (fix: ...) na mesma branch feature/backend-import-initial, antes do merge em develop.

5.1 Alteracoes necessarias

1	Repositories/Interfaces/IClienteRepository.cs	Adicionar parametro <code>usuarioId</code> a assinatura de <code>GetByCpfAsync</code>
2	Repositories/ClienteRepository.cs	Filtrar query por <code>c.UsuarioId == usuarioId</code> em adicao ao CPF
3	Services/ClienteService.cs (linhas 33 e 67)	Passar <code>usuarioId</code> nas duas chamadas a <code>GetByCpfAsync</code>
4	Data/AppDbContext.cs	Trocar <code>HasIndex(e => e.Cpf).IsUnique()</code> por <code>HasIndex(e => new { e.UsuarioId, e.Cpf }).IsUnique()</code>

5.2 Justificativa do trade-off

Apos o fix, dois tenants diferentes poderao ter o mesmo CPF cadastrado em suas respectivas carteiras de clientes. Isso eh **correto** no modelo de negocio: o mesmo CPF pode ser cliente de mais de um microempreendedor, e cada tenant gerencia sua propria base de forma isolada. A unicidade composta (`UsuarioId`, `Cpf`) mantem a consistencia dentro do tenant sem vaziar informacao entre tenants.

5.3 Plano de execucao

- 1 Aplicar as 4 mudancas na branch `feature/backend-import-initial` via Edit.
- 2 Gerar migration EF Core para o novo indice composto (sera feita junto com a migration inicial do banco, na fase do Supabase).
- 3 Commit: `fix: filtrar GetByCpfAsync por UsuarioId` (security review PR #1)
- 4 Push para a mesma branch.
- 5 Re-rodar `/security-review` sobre o diff atualizado para confirmar que o achado foi resolvido.
- 6 Aprovar e mergear o PR em `develop`.

Documento gerado automaticamente como parte do bootstrap de infraestrutura do projeto Micro-ERP Auto.